Métodos Cuantitativos

Clase 07 - Distribuciones continuas

Unidad I: Estadística y Probabilidades con Python

Breve repaso

- Hasta el momento, hemos estudiado algunas **distribuciones discretas** y hemos utilizado la **función de masa de probabilidad (PMF)** y la **función de distribución acumulativa (CDF)** para calcular probabilidades puntuales o rangos específicos.
- A continuación, estudiaremos las **distribuciones continuas**, que nos permiten modelar variables capaces de tomar un número infinito de valores dentro de un intervalo determinado.

Distribuciones continuas

- En el estudio de las distribuciones continuas, la **función de densidad de probabilidad (PDF)** reemplaza a la **PMF** como la herramienta principal para describir la probabilidad relativa de que una variable aleatoria tome un valor específico.
- Aunque, en el sentido estricto, la probabilidad de que una variable continua tome un valor puntual exacto es cero, la **PDF nos permite determinar la densidad de probabilidad en cualquier punto del dominio de la variable**, ofreciendo una forma de entender cómo se distribuyen los valores a lo largo de un continuo.

Digresión

- ¿Por qué la probabilidad de que una variable continua tome un valor puntual exacto es cero?
- En las **distribuciones discretas**, tratamos con variables que pueden tomar **valores contables**.
- Por ejemplo, el número de autos que pasan por un peaje en una hora puede ser 0, 1, 2, 3, etc., pero no puede ser 2.5.
- La probabilidad de cada uno de estos valores contables puede ser descrita por la función de masa de probabilidad (PMF), y sumando estas probabilidades individuales podemos obtener la probabilidad total.

Digresión

Distribuciones continuas

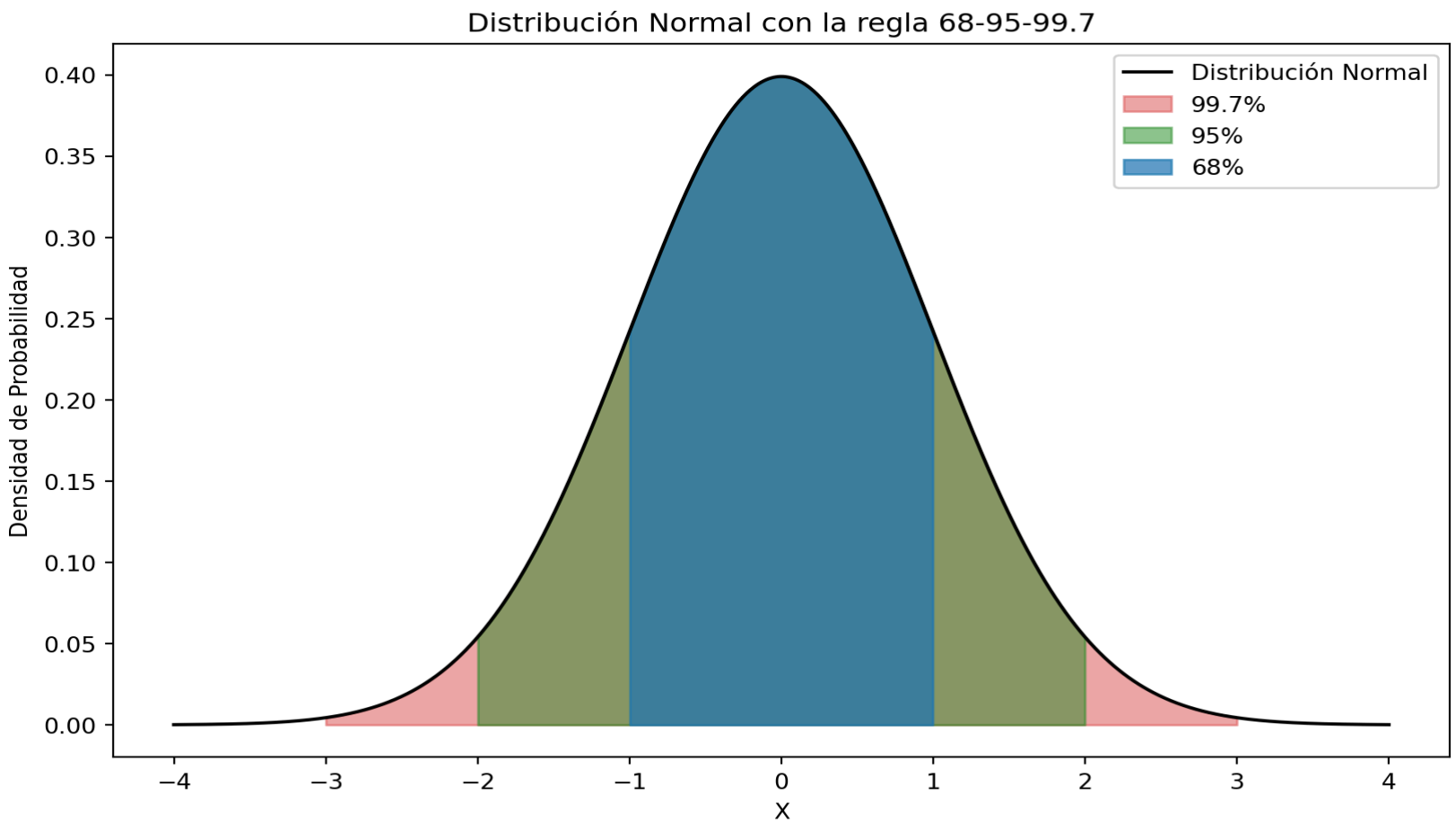
- Seguiremos empleando la **función de distribución acumulativa (CDF)**, la cual es igualmente fundamental en el contexto de las distribuciones continuas.
- La CDF nos indica la probabilidad de que una variable aleatoria tome un **valor menor o igual a un cierto límite**, proporcionando una forma directa de **calcular rangos de probabilidad**.

¿Qué veremos hoy?

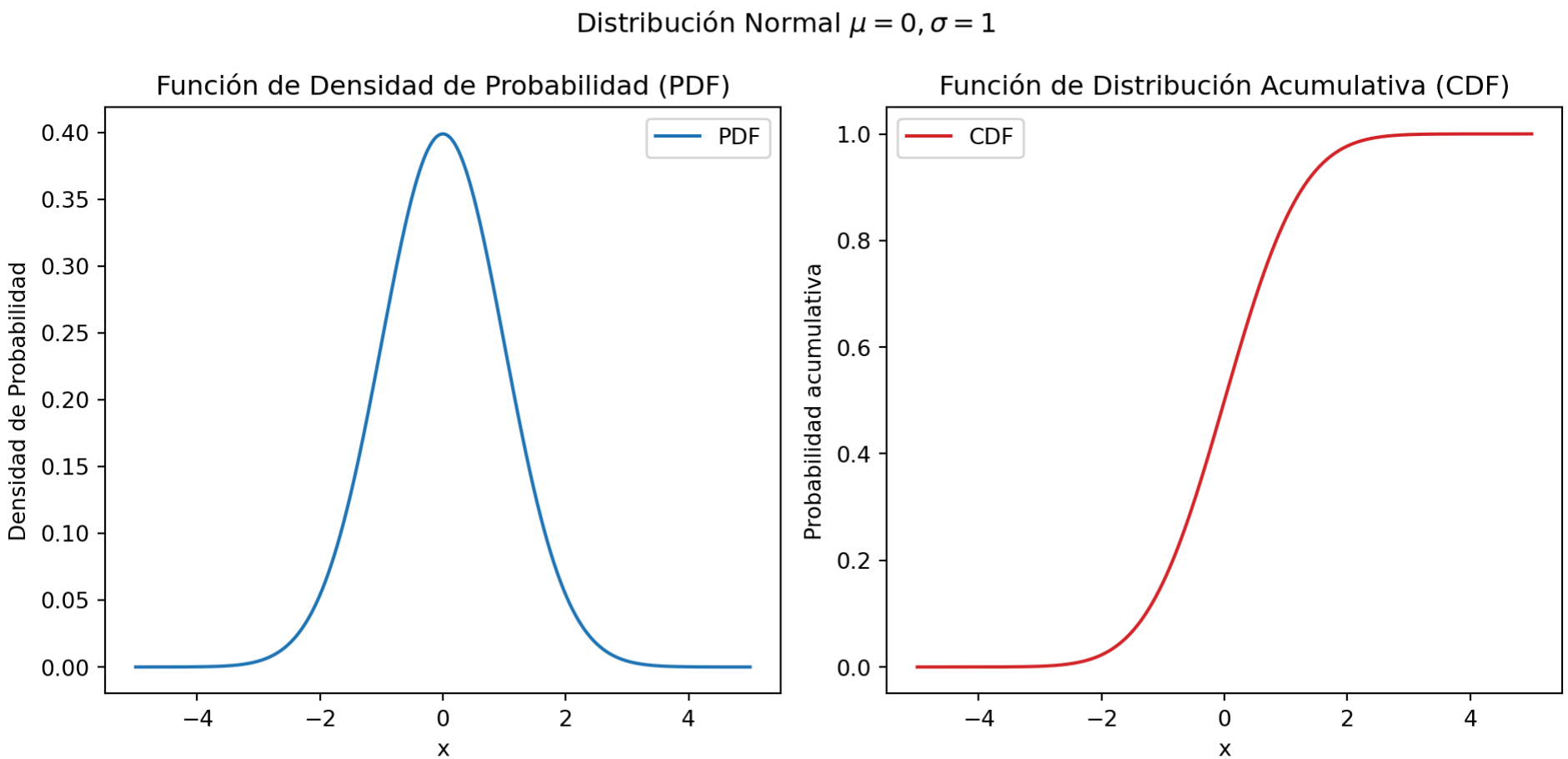
- **Distribución normal**
- Normal estándar
- Uniforme continua
- Exponencial y gamma
- Teorema del Límite Central

Distribución Normal

Distribución Normal



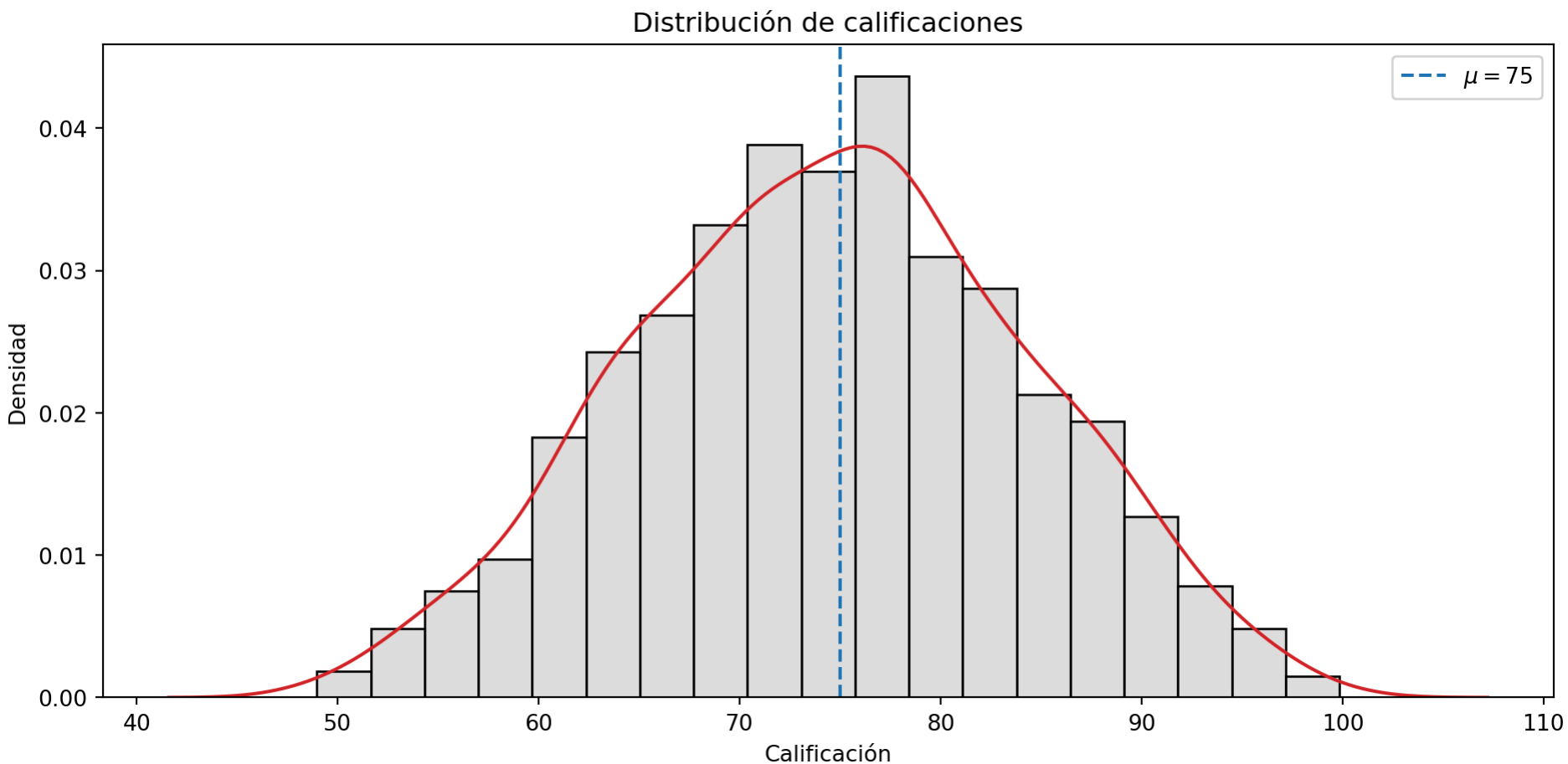
Distribución Normal PDF y CDF



Distribución Normal

- Supongamos que realizamos una prueba a 100 alumnos, en la cual el puntaje máximo es 100.
- Al analizar los resultados de las pruebas, observamos que las calificaciones se **distribuyen normalmente** con una **media (μ) de 75 puntos** y **una desviación estándar (σ) de 10 puntos**.
- Nos gustaría estimar **la probabilidad de que un estudiante elegido al azar haya obtenido una calificación dentro de un rango específico**, así como **la probabilidad de obtener una calificación superior a un cierto valor**.

Distribución Normal



Distribución Normal

- Para resolver este problema, podemos usar la **distribución normal** para modelar las calificaciones de los alumnos.
- La variable aleatoria X representa la calificación de un alumno.
- La media μ es de 75 puntos y la desviación estándar σ es de 10 puntos.
- Con estos datos, podemos calcular, por ejemplo:
  - La probabilidad de que un alumno obtenga una calificación entre 65 y 85 puntos.
  - La probabilidad de que un alumno obtenga una calificación superior a 90 puntos.
  - La densidad de la distribución en 75 puntos y cómo interpretarla correctamente.

Distribución Normal en Python

La probabilidad de que un alumno obtenga una calificación entre 65 y 85 puntos:

```python
from scipy.stats import norm

mu = 75
sigma = 10

# calculamos la probabilidad de obtener una calificación de 85 o menos
prob_85 = norm.cdf(85, mu, sigma)

# calculamos la probabilidad de obtener una calificación de 65 o menos
prob_65 = norm.cdf(65, mu, sigma)

# calculamos la probabilidad de obtener una calificación entre 65 y 85
prob_65_85 = prob_85 - prob_65

# reportamos la probabilidad
print(f"{prob_65_85} o {round(prob_65_85 * 100, 2)}%")
```

```
0.6826894921370859 o 68.27%
```

Distribución Normal en Python

La probabilidad de que un alumno obtenga una calificación superior a 90 puntos:

```python
from scipy.stats import norm

mu = 75
sigma = 10

# calculamos la probabilidad de obtener una calificación superior a 90
prob_gt_90 = norm.sf(90, mu, sigma)

# reportamos la probabilidad
print(f"{prob_gt_90} o {round(prob_gt_90 * 100, 2)}%")
```

```
0.06680720126885806 o 6.68%
```

```python
# calculamos la probabilidad de obtener una calificación de 90 o menos
prob_90 = norm.cdf(90, mu, sigma)
prob_gt_90 = 1 - prob_90

```

```python
print(f"{prob_gt_90} o {round(prob_gt_90 * 100, 2)}%")
```

```
0.06680720126885809 o 6.68%
```

Distribución Normal en Python

En una distribución continua, la probabilidad de que un alumno obtenga exactamente 75 puntos es 0. Sin embargo, sí podemos calcular la **densidad** de la distribución en 75 puntos.

```python
from scipy.stats import norm

mu = 75
sigma = 10

# en una distribución continua, P(X = 75) = 0; aquí calculamos la densidad
densidad_75 = norm.pdf(75, mu, sigma)

# reportamos la densidad
print(f"Densidad en 75 puntos: {densidad_75}")
```

```
Densidad en 75 puntos: 0.03989422804014327
```

¿Qué veremos hoy?

- Distribución normal
- **Normal estándar**
- Uniforme continua
- Exponencial y gamma
- Teorema del Límite Central

Distribución Normal Estándar

- La distribución normal estándar es un caso específico de la distribución normal, una de las distribuciones continuas más fundamentales y ampliamente utilizadas en estadística y teoría de probabilidades.
- La principal característica de esta distribución es que tiene una **media** (μ) **= 0** y la **desviación estándar** (σ) **= 1**.
- Esto significa que hemos “estandarizado” la distribución normal general para tener una media de 0 y una desviación estándar de 1.

Distribución Normal Estándar

La función de densidad de probabilidad de la distribución normal estándar es:

$$f(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}z^2}$$

donde z es una variable **estandarizada** que se obtiene a partir de la ecuación:

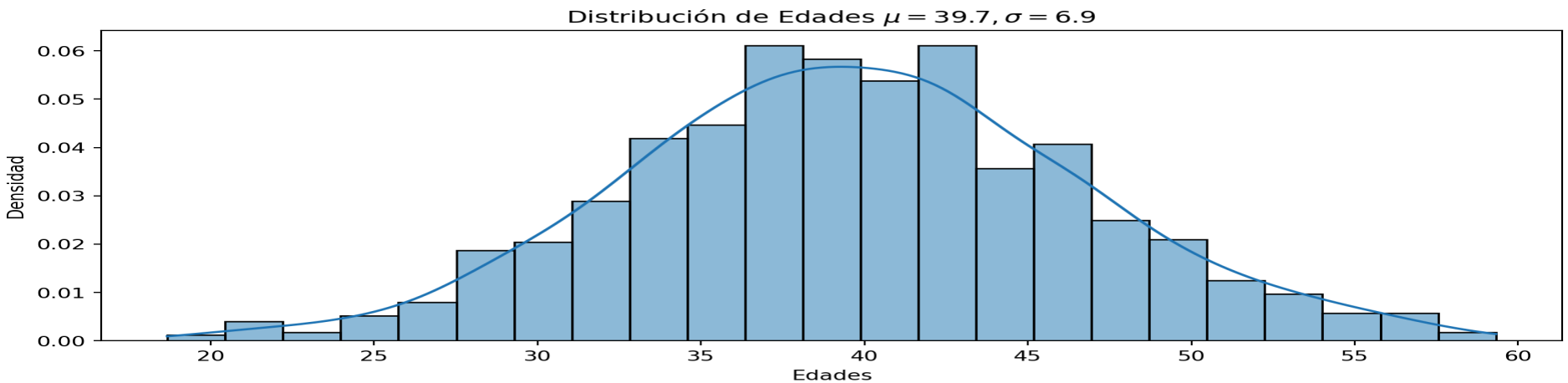
$$z = \frac{x - \mu}{\sigma}$$

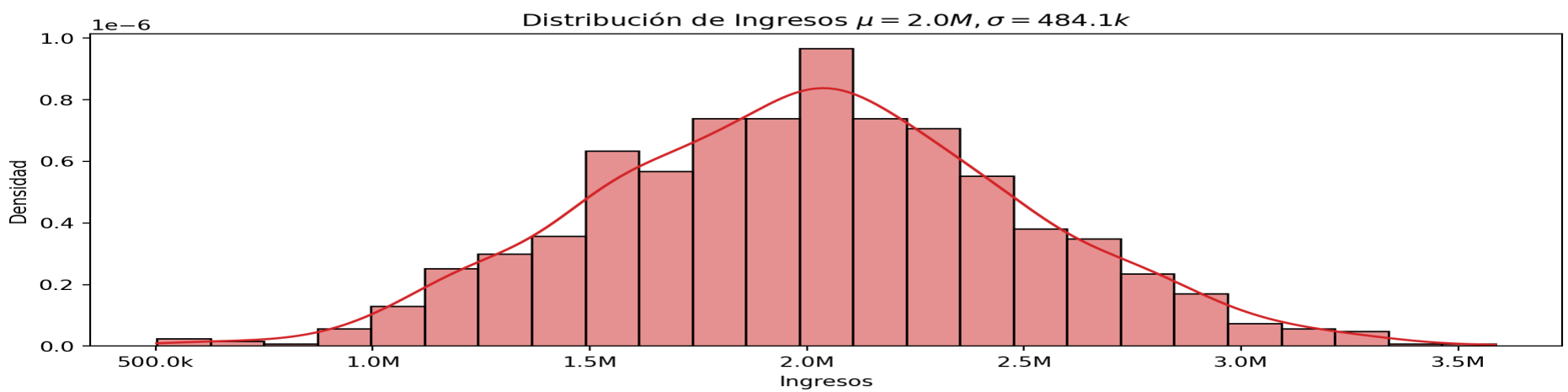
Este proceso de estandarización **transforma cualquier distribución normal en la distribución normal estándar**. La variable z representa el número de desviaciones estándar que un valor x está por encima o por debajo de la media μ.

Distribución Normal Estándar

- La normalización permite comparar puntuaciones o datos de diferentes distribuciones normales, facilitando la interpretación de estadísticas en una variedad de campos.
- Para entender su importancia, veamos el siguiente ejemplo:
  - Supongamos que cierto banco quiere crear 4 segmentos de clientes o *clusterizar* (concepto utilizado en machine learning para separar o categorizar a un grupo de datos de acuerdo a algunas características en común), para realizar esta tarea, el banco dispone la información de la **edad y los ingresos de un grupo de 1000 clientes**.

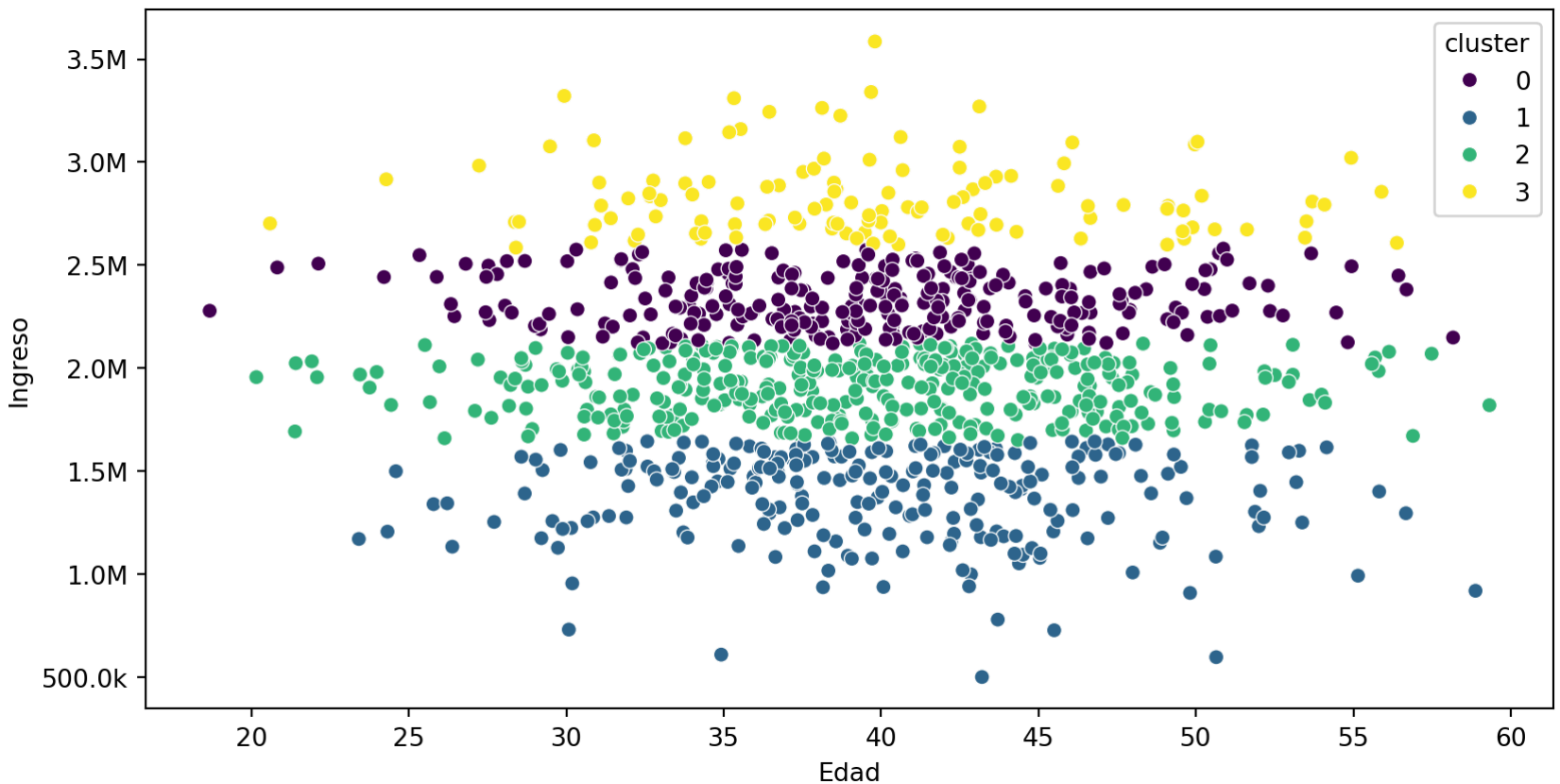
Distribución Normal Estándar





Distribución Normal Estándar

Al aplicar el algoritmo *k-medias* a los datos, obtenemos los siguientes resultados: (El algoritmo k-medias agrupa datos en *k* grupos asignando iterativamente puntos al centroide más cercano y actualizando los centroides hasta converger).



El resultado del algoritmo utilizando los datos de ingreso y edad **sin normalizar**, tiende a separar los registros principalmente según los ingresos, lo cual se debe a que la media del ingreso es mayor que la de la edad.

Distribución Normal Estándar

Recordemos la fórmula:

$$z = \frac{x - \mu}{\sigma}$$

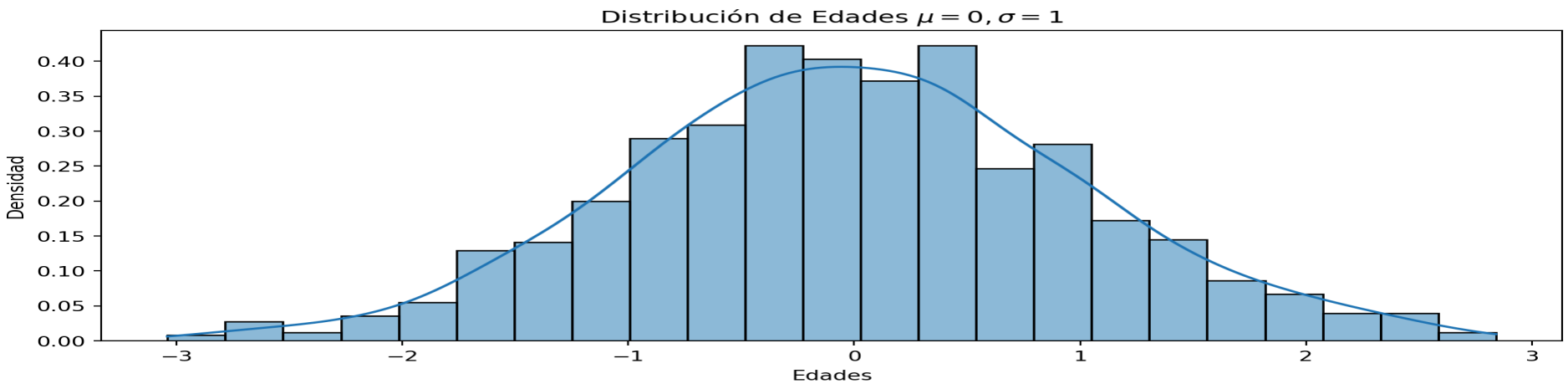
La podemos aplicar de la siguiente manera:

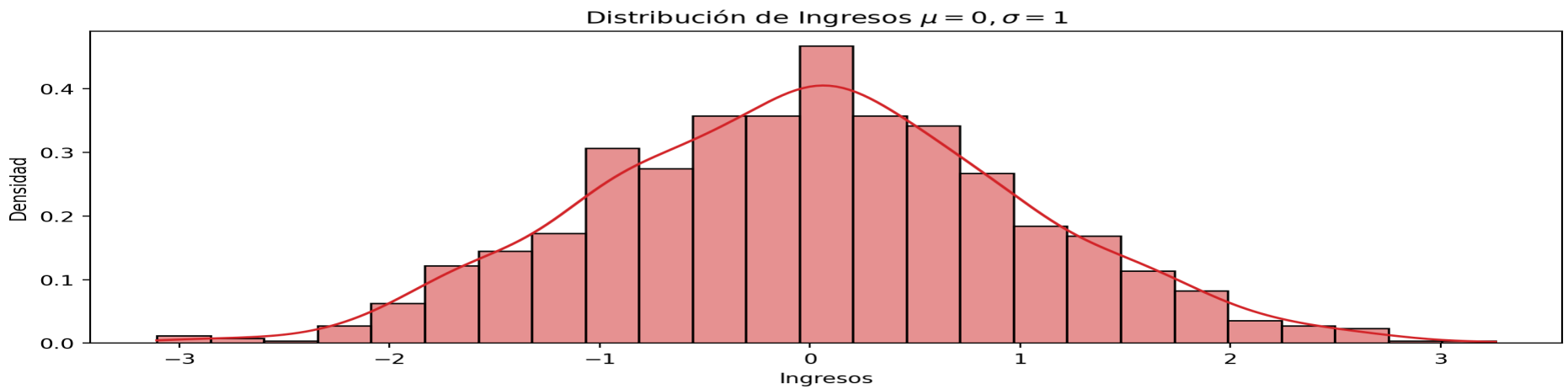
```
data["zscore_edad"] = (
  (data["Edad"] - data["Edad"].mean()) / data["Edad"].std()
)

data["zscore_ingreso"] = (
  (data["Ingreso"] - data["Ingreso"].mean()) / data["Ingreso"].std()
)

print(data.head())
```

```
        Edad  Ingreso  zscore_edad  zscore_ingreso
0  52.348366  2277981     1.832162        0.559902
1  42.801100  2446236     0.451040        0.907305
2  46.851166  1788842     1.036928       -0.450043
```

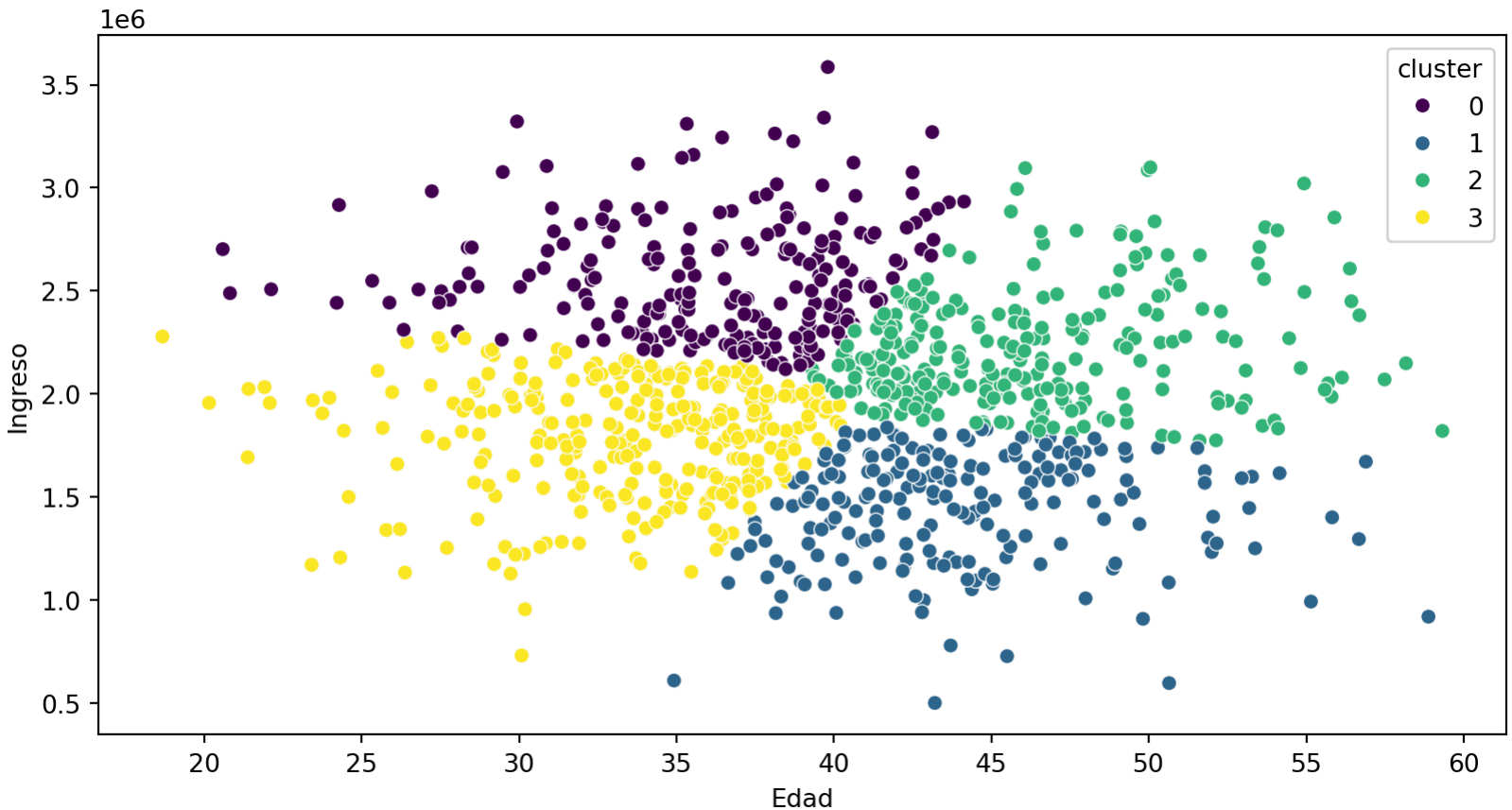
```
4  53.072906  2114026     1.936974        0.221377
```

Distribución Normal Estándar





Distribución Normal Estándar



- Normalizar las variables antes de aplicar el algoritmo k-medias asegura que cada una tenga el mismo peso durante la formación de los *clusters*.
- Si las variables tienen diferentes rangos, la que tenga valores más grandes dominará la medida de distancia, lo que podría resultar en una segmentación sesgada.
- Al normalizar, se ayuda a que la formación de los segmentos sea más homogénea y refleje mejor las similitudes en términos de la estructura de los datos en lugar de su escala.

¿Qué veremos hoy?

- Distribución normal
- Normal estándar
- **Uniforme continua**
- Exponencial y gamma
- Teorema del Límite Central

Distribución Uniforme Continua

- La distribución uniforme continua es una distribución continua que asigna la misma **densidad** a todos los valores dentro de un intervalo específico definido por los límites a y b.
- Es útil para modelar situaciones donde cualquier valor real dentro de un rango es igualmente plausible, como tiempos de llegada, mediciones continuas o simulaciones de fenómenos naturales o procesos industriales.

Distribución Uniforme Continua

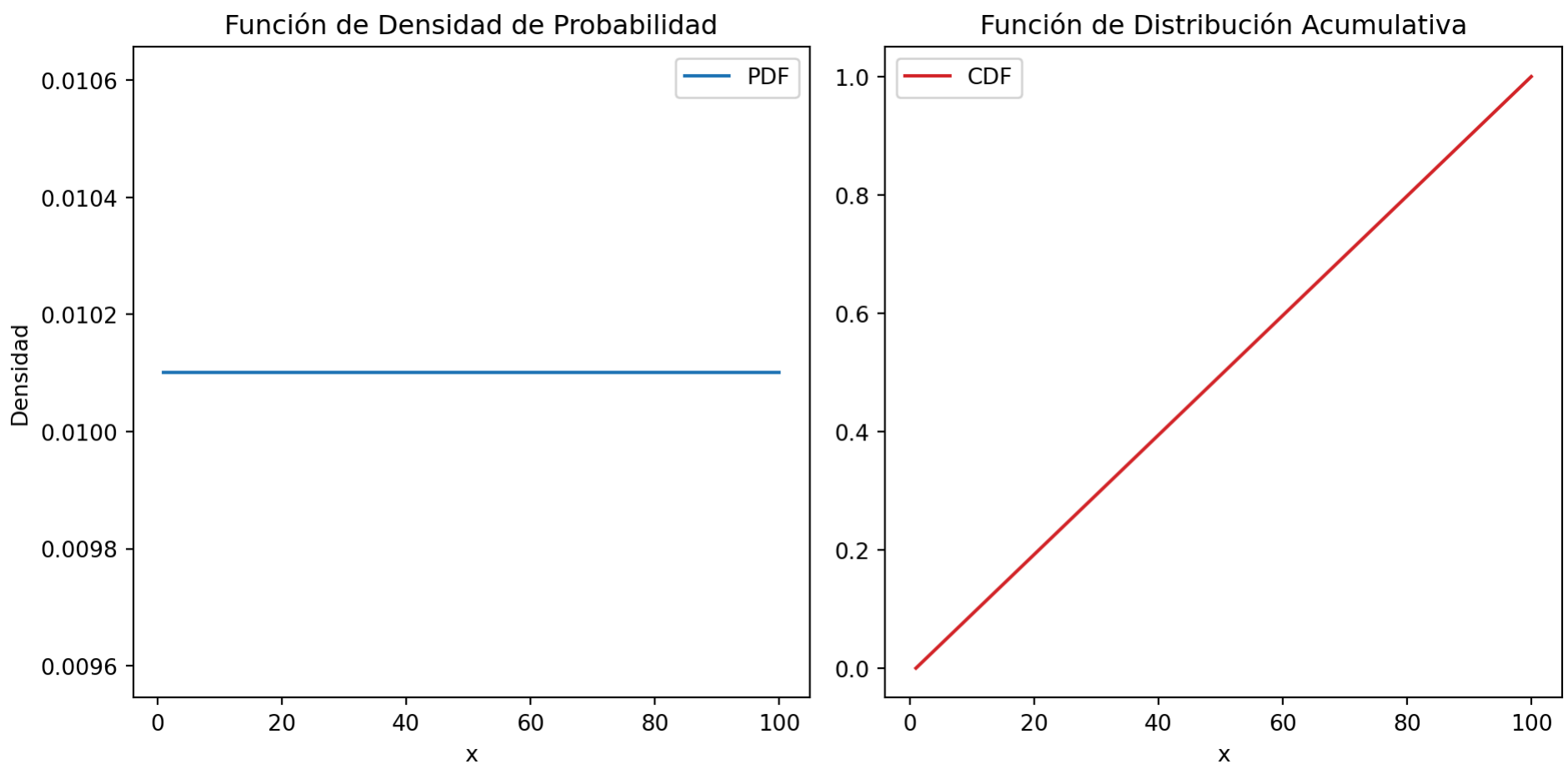
- Imaginemos que una máquina genera un valor real al azar entre 1 y 100. Nos interesa calcular la probabilidad de que el valor generado caiga dentro de ciertos rangos.
- Para analizar esta situación, podemos utilizar la **distribución uniforme continua**.
  - La variable aleatoria X representa el valor real generado por la máquina.
  - Los parámetros a y b definen los límites de la distribución, que en este caso son 1 y 100, respectivamente.
- Dado que es una distribución uniforme continua, todos los valores del intervalo tienen la misma densidad. Por ejemplo, podemos calcular:
  - La probabilidad de que el valor generado sea menor que 50.
  - La probabilidad de que el valor esté entre 25 y 75.

Distribución Uniforme en Python

```python
from scipy.stats import uniform

# definir los límites de la distribución
a = 1  # límite inferior
b = 100  # límite superior

# crear una distribución uniforme continua entre a y b
distribucion_uniforme = uniform(a, b-a)

# calcular probabilidades
# probabilidad de que el valor generado sea menor que 50
prob_lt_50 = distribucion_uniforme.cdf(50)

# probabilidad de que el valor esté entre 25 y 75
prob_entre_25_75 = distribucion_uniforme.cdf(75) - distribucion_uniforme.cd

print(f"Probabilidad de que el valor sea menor que 50: {prob_lt_50}")
print(f"Probabilidad de que el valor esté entre 25 y 75: {prob_entre_25_75}
```

```
Probabilidad de que el valor sea menor que 50: 0.494949494949495
Probabilidad de que el valor esté entre 25 y 75: 0.5050505050505051
```

Distribución Uniforme PDF y CDF



¿Qué veremos hoy?

- Distribución normal
- Normal estándar
- Uniforme continua
- **Exponencial y gamma**
- Teorema del Límite Central

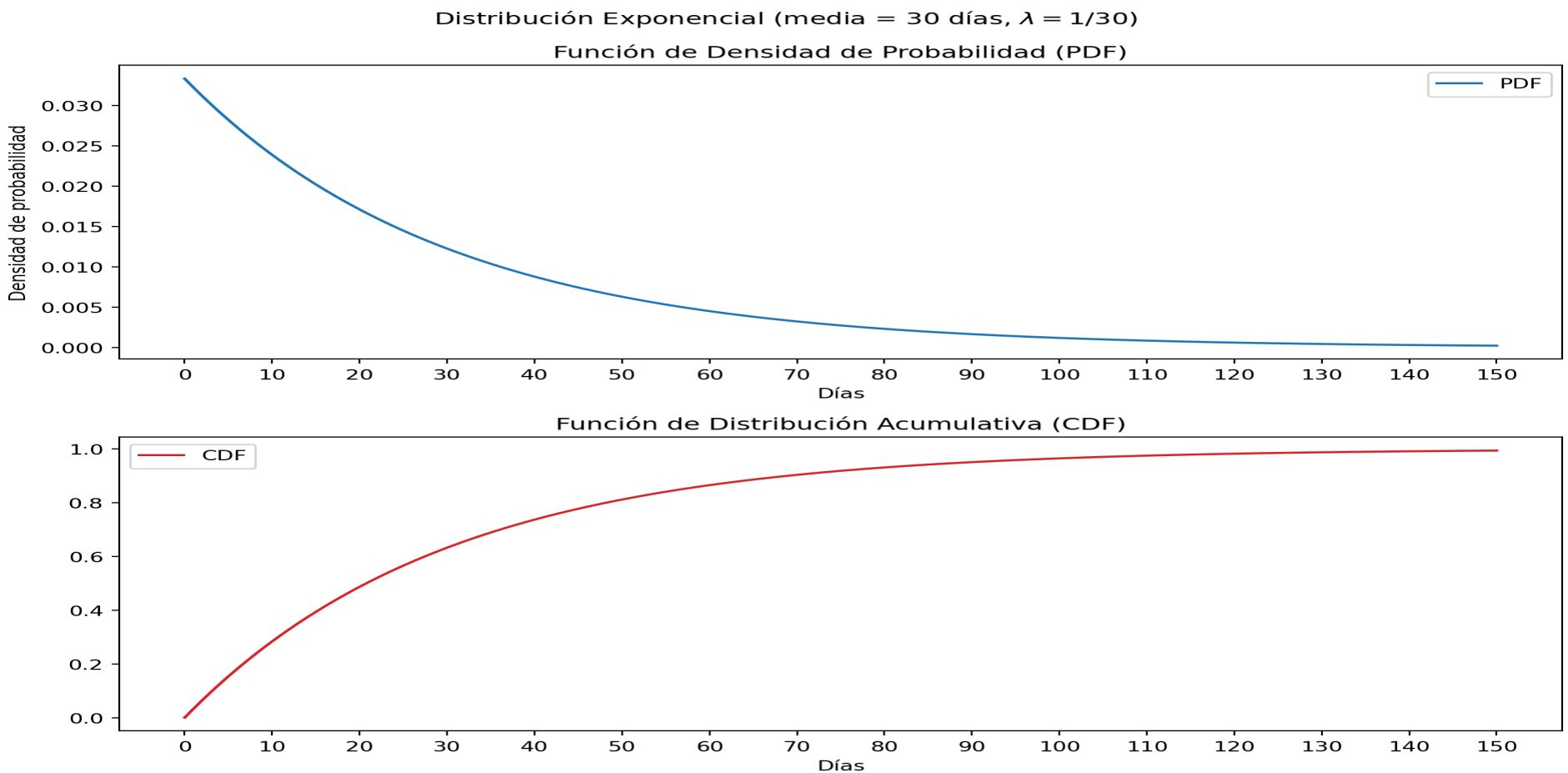
Distribución Exponencial

- La distribución Exponencial es una distribución de probabilidad continua que se utiliza comúnmente para modelar el tiempo entre eventos en un proceso de **Poisson**, es decir, cuando los eventos ocurren de manera continua e independiente a una tasa constante.
- Es particularmente útil para modelar tiempos de espera, como la duración entre fallos de equipos, tiempos de respuesta en sistemas de servicio, y otros eventos que siguen un patrón de tiempo continuo.

Distribución Exponencial

- Consideremos el caso de una compañía de telecomunicaciones que desea estudiar el tiempo entre fallas de su red para mejorar el servicio. A partir de registros previos, se determina que el tiempo promedio entre fallos es de 30 días.
- Para analizar y prever estos tiempos entre fallos, podemos emplear la **distribución Exponencial**.
  - La variable aleatoria X representa el tiempo entre fallos consecutivos en la red.
  - El parámetro λ, la tasa de fallos, se calcula como el inverso del tiempo medio entre fallos, lo que en este caso es $\frac{1}{30}$.
- Por ejemplo, podemos calcular:
  - La probabilidad de que el siguiente fallo ocurra en menos de 10 días.
  - La probabilidad de que el siguiente fallo ocurra entre 20 y 30 días.
  - La densidad de la distribución en 25 días y cómo interpretarla correctamente.

Distribución Exponencial PDF y CDF



Distribución Exponencial en Python

La probabilidad de que el siguiente fallo ocurra en menos de 10 días:

```python
from scipy.stats import expon

# parámetro de la distribución exponencial
lambda_inv = 30  # tiempo medio entre fallos

# probabilidad de que el siguiente fallo ocurra en menos de 10 días
prob_menos_10 = expon.cdf(10, scale=lambda_inv)

print(f"Prob. que el siguiente fallo ocurra en menos de 10 días: {prob_menc
```

```
Prob. que el siguiente fallo ocurra en menos de 10 días: 0.28346868942621073
```

Distribución Exponencial en Python

La probabilidad de que el siguiente fallo ocurra entre 20 y 30 días:

```python
from scipy.stats import expon

# parámetro de la distribución exponencial
lambda_inv = 30  # tiempo medio entre fallos

# probabilidad de que el siguiente fallo ocurra entre 20 y 30 días
prob_menos_30 = expon.cdf(30, scale=lambda_inv)
prob_menos_20 = expon.cdf(20, scale=lambda_inv)
prob_bt_20_30 = prob_menos_30 - prob_menos_20

print(f"Prob. de que el siguiente fallo ocurra entre 20 y 30 días: {prob_bt
```

```
Prob. de que el siguiente fallo ocurra entre 20 y 30 días: 0.14553767786114968
```

Distribución Exponencial en Python

En una distribución continua, la probabilidad de que el siguiente fallo ocurra exactamente a 25 días es 0. Sin embargo, sí podemos calcular la **densidad** en 25 días.

```python
from scipy.stats import expon

# parámetro de la distribución exponencial
lambda_inv = 30  # tiempo medio entre fallos

# en una distribución continua, P(X = 25) = 0; aquí calculamos la densidad
densidad_25 = expon.pdf(25, scale=lambda_inv)

print(f"Densidad en 25 días: {densidad_25}")
```

```
Densidad en 25 días: 0.01448660695023594
```

Distribución Exponencial en Python

La probabilidad de que pasen más de 50 días antes del siguiente fallo:

```python
from scipy.stats import expon

# parámetro de la distribución exponencial
lambda_inv = 30  # tiempo medio entre fallos

# probabilidad de que pasen más de 50 días antes del siguiente fallo
prob_gt_50 = expon.sf(50, scale=lambda_inv)

print(f"Prob. de que pasen más de 50 días antes del siguiente fallo: {prob_
```

```
Prob. de que pasen más de 50 días antes del siguiente fallo: 0.18887560283756183
```

Distribución Gamma

- La distribución Gamma es una distribución continua que se utiliza para **modelar tiempos de espera hasta que ocurren k eventos en un proceso con tiempos entre eventos siguiendo una distribución exponencial**.
- Parámetros de la Distribución Gamma:
  - Parámetro de Forma (k): El parámetro de forma, influye en la forma de la distribución de la variable aleatoria.
  - Parámetro de Escala (θ): El parámetro determina la escala de la distribución y está relacionado con el tiempo promedio que se requiere para completar cada fase del proyecto.
- Esta distribución es especialmente útil para modelar la duración de eventos en una amplia gama de aplicaciones

Distribución Gamma

- Imaginemos que estamos evaluando el tiempo total que toman los desarrollos de software en ser completados en cierta empresa.
- Basados en datos históricos, sabemos que, en promedio, un proyecto se completa en 120 días, pero la variabilidad es alta debido a la naturaleza compleja de estos proyectos.
- Para modelar el tiempo de finalización de estos proyectos, podemos utilizar la **distribución Gamma**.

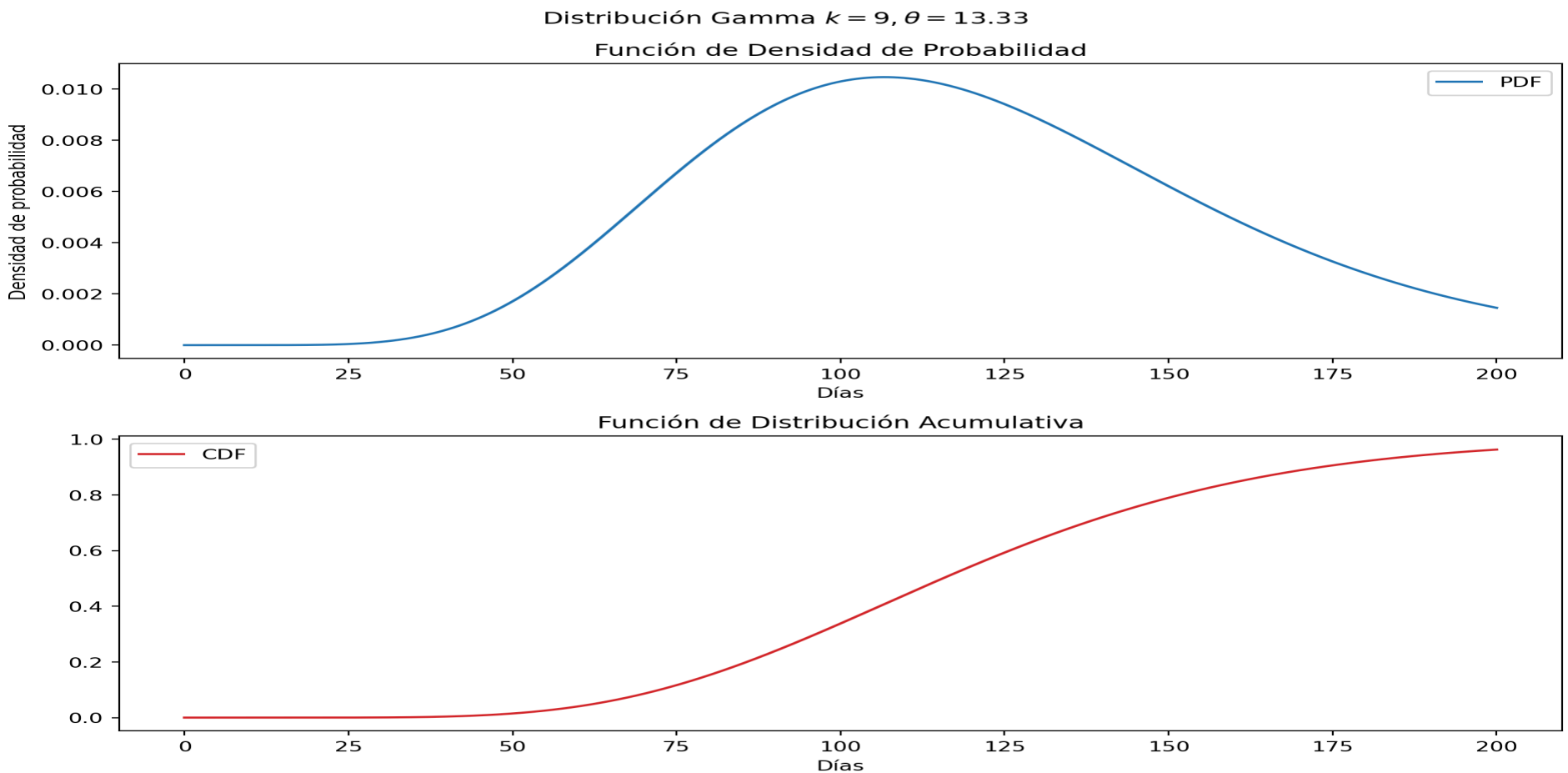
Distribución Gamma

Distribución Gamma en Python

```python
from scipy.stats import gamma

# parámetros de la distribución Gamma
k = 9  # parámetro de forma
media_completar_proyecto = 120
theta = (media_completar_proyecto/k)  # parámetro de escala

# crear una distribución Gamma
distribucion_gamma = gamma(a=k, scale=theta)

# calcular probabilidades
# probabilidad de que un proyecto se complete en menos de 100 días
prob_lt_100 = distribucion_gamma.cdf(100)

# probabilidad de que un proyecto tome más de 150 días para completarse
prob_gt_150 = distribucion_gamma.sf(150)

print(f"Prob. que un proyecto se complete en menos de 100 días: {prob_lt_10
```

```
Prob. que un proyecto se complete en menos de 100 días: 0.3380328808585169
Prob. que un proyecto tome más de 150 días para completarse: 0.21054094346059704
```

Distribución Gamma PDF y CDF



Distribuciones Discretas y Continuas

Característica	Distribución Discreta
Valores posibles	Contables (finitos o infinitos)
Ejemplos	Número de caras al lanzar una moneda, número de hi
Representación gráfica	Función de masa de probabilidad (PMF)
Cálculo de probabilidad	Sumando probabilidades en puntos específicos

Análisis de medias de distribuciones

- Consideremos una variable aleatoria X proveniente de una distribución arbitraria; por ejemplo, una distribución Binomial y realicemos el siguiente experimento:
  - Establecemos $X \sim Binomial(10, 0.5)$.
  - Generamos 2000 muestras, cada una de tamaño 40, lo que implica realizar 2000 repeticiones de 40 experimentos binomiales, cada uno con 10 ensayos y una probabilidad de éxito de 0.5.

```python
from scipy.stats import binom

# parámetros
numero_muestras = 2000
tamano_muestra = 40

# crear distribución
distribucion_binom = binom(n=10, p=0.5)

# crear variable aleatoria
```

```python

print(va)
```

```
[[5 5 3 ... 7 6 5]
 [5 6 7 ... 4 6 4]
 [6 5 5 ... 5 3 6]
 ...
 [6 6 5 ... 7 5 5]
 [6 3 5 ... 6 4 7]
 [6 5 6 ... 4 2 3]]
```

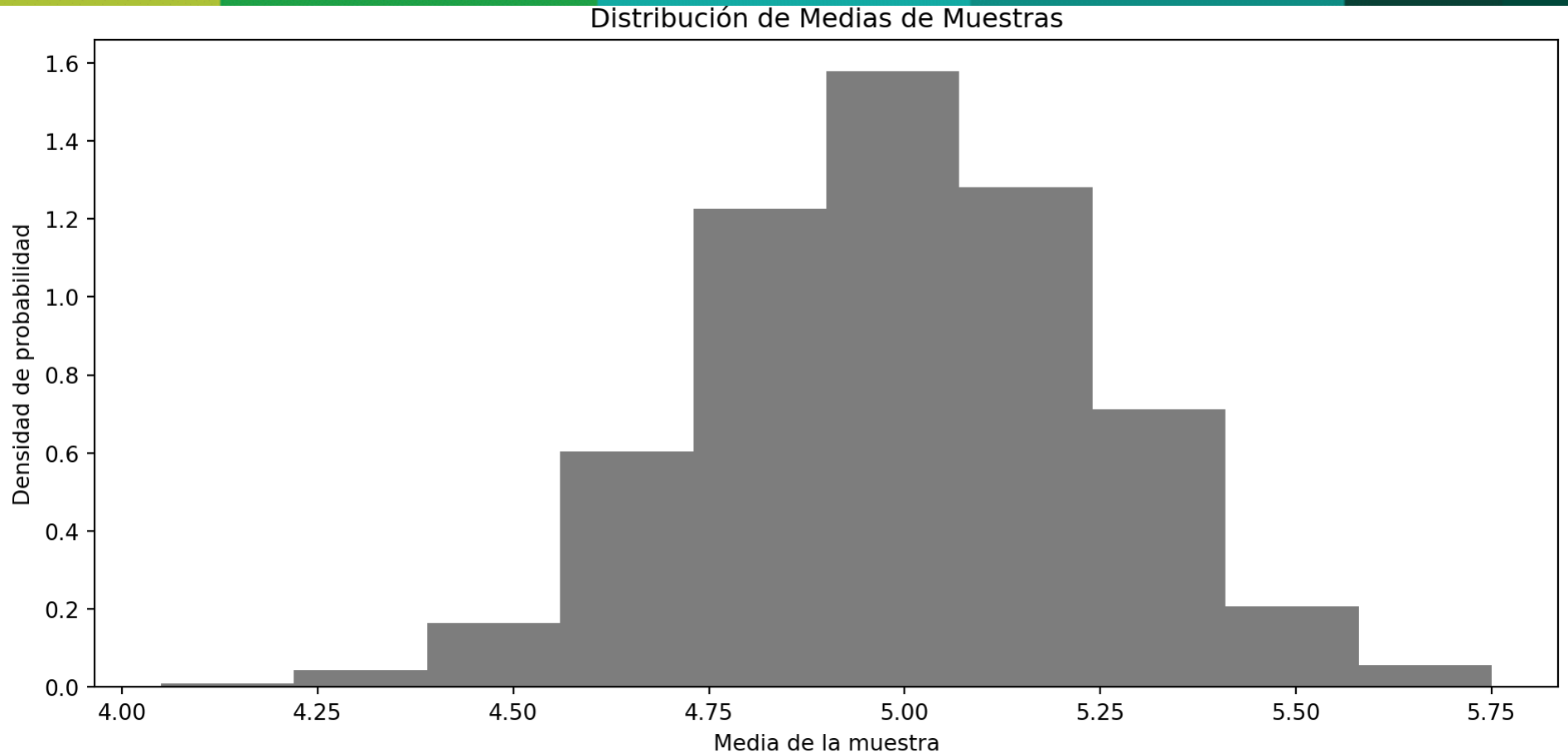
Análisis de medias de distribuciones

- Calculemos las medias de cada una de las 2000 repeticiones, obteniendo así un conjunto de 2000 medias:

```
medias = np.mean(va, axis=1)

print(len(medias))
print(medias)
```

```
2000
[5.2   5.1   5.3   ... 5.225 5.05  4.925]
```

- A continuación, visualicemos estas medias en un histograma para entender su distribución:


Distribución de Medias de Muestras
Densidad de probabilidad
Media de la muestra
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1.6
4.00
4.25
4.50
4.75
5.00
5.25
5.50
5.75

¿Qué veremos hoy?

- Distribución normal
- Normal estándar
- Uniforme continua
- Exponencial y gamma
- **Teorema del Límite Central**

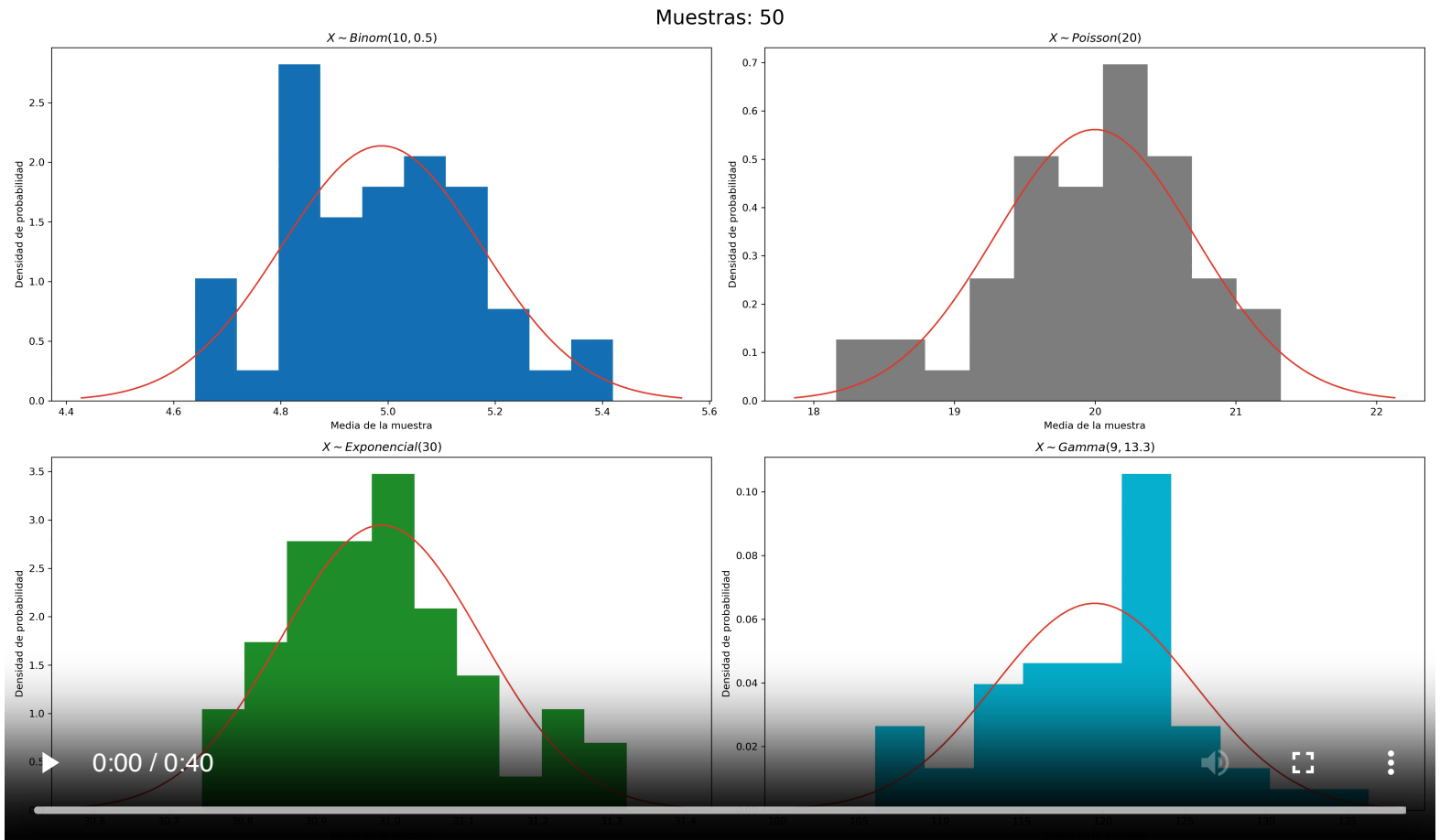
Teorema del Límite Central

- El comportamiento anteriormente observado constituye un pilar central de la estadística y se denomina **Teorema del Límite Central (TLC)**.
- Este teorema es crucial porque nos ayuda a entender por qué la distribución normal es tan prevalente en estadística y por qué funciona bien para aproximaciones y análisis estadísticos, incluso cuando la distribución subyacente de los datos no es normal.

Teorema del Límite Central

- El Teorema del Límite Central establece que, para una muestra suficientemente grande de observaciones independientes e idénticamente distribuidas, la distribución de la media muestral tiende a una distribución normal, independientemente de la forma de la distribución original.
- Este teorema tiene algunas condiciones y suposiciones:
  - Las observaciones deben ser independientes.
  - Las observaciones deben provenir de una distribución con varianza finita.
  - El tamaño de la muestra debe ser suficientemente grande (usualmente se considera un tamaño de muestra de 30 o más como suficiente).

Teorema del Límite Central

- El TLC es fundamental en estadística porque justifica la utilización de la inferencia estadística, que incluye la construcción de intervalos de confianza y la realización de pruebas de hipótesis.
- Gracias al TLC, podemos usar métodos estadísticos basados en la distribución normal para modelar fenómenos y tomar decisiones informadas en situaciones donde los datos reales no siguen necesariamente esta distribución.

Teorema del Límite Central